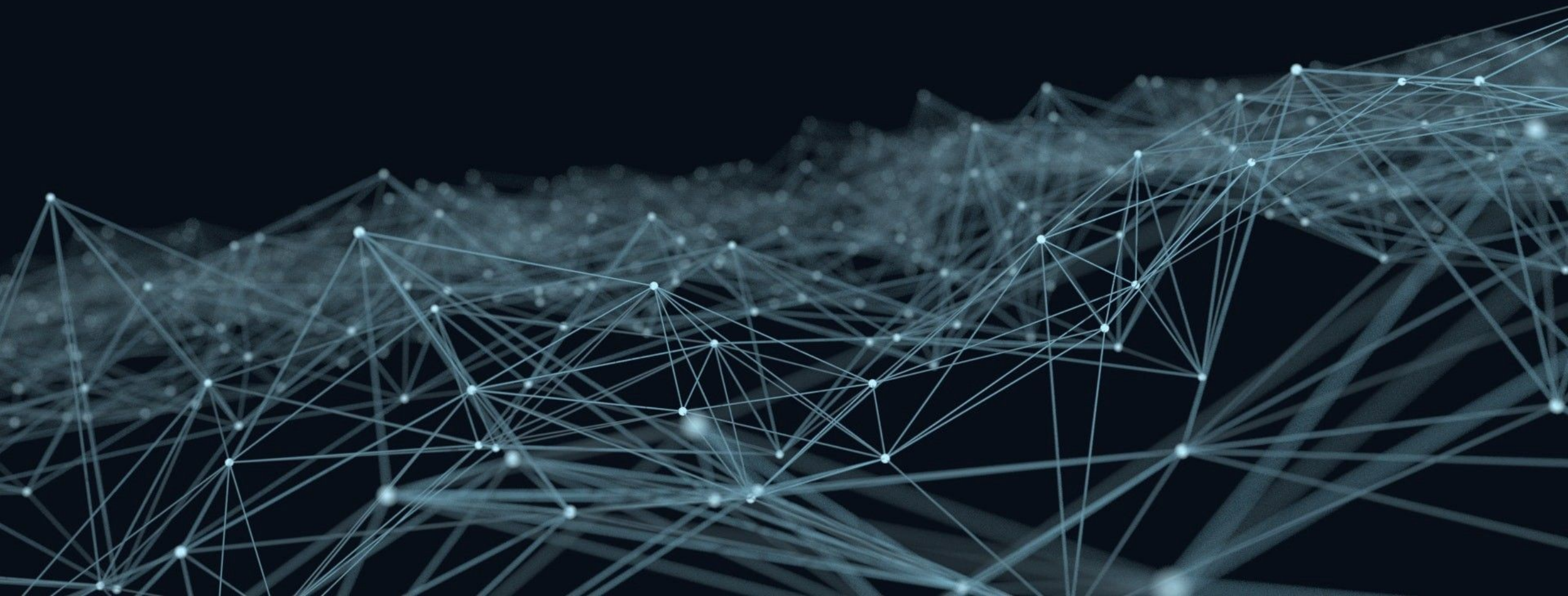
Java Full Stack Development

2. Indexes, Triggers and Partitions



Module Topics

- Triggers
- PS/SQL Use Cases
- Indexes
- Partitions

Triggers

Triggers

- A trigger
  - is a PL/SQL block associated with a table, view, or schema event that Oracle executes automatically when a specified event occurs
  - The developer does not call a trigger
    - Oracle fires it in response to DML (INSERT, UPDATE, DELETE) or DDL (CREATE, ALTER, DROP) operations
  - Triggers are invisible to the application performing the DML
  - They execute as an extension of the triggering statement's transaction

The four trigger timing and event combinations

Timing	Event	Fires
BEFORE	INSERT, UPDATE, DELETE	Before the row is modified -- can alter the new values
AFTER	INSERT, UPDATE, DELETE	After the row is modified -- cannot alter the values; used for auditing
INSTEAD OF	INSERT, UPDATE, DELETE	On a view -- replaces the DML with custom logic
BEFORE/ AFTER	DDL, LOGON, LOGOFF	Schema-level events; used for administrative control

Triggers

- Triggers exist to
  - Enforce complex business rules that cannot be expressed as a simple constraint
  - Maintain audit trails automatically, regardless of which application or process modifies the data
  - Enforce derived data that must be kept consistent with source data
  - Implement row-level security or validation that depends on the values being changed

The four trigger timing and event combinations

Timing	Event	Fires
BEFORE	INSERT, UPDATE, DELETE	Before the row is modified -- can alter the new values
AFTER	INSERT, UPDATE, DELETE	After the row is modified -- cannot alter the values; used for auditing
INSTEAD OF	INSERT, UPDATE, DELETE	On a view -- replaces the DML with custom logic
BEFORE/ AFTER	DDL, LOGON, LOGOFF	Schema-level events; used for administrative control

Triggers

- Row-level versus statement-level triggers
  - FOR EACH ROW: fires once per affected row
    - Has access to :OLD and :NEW values
    - :OLD - the values the row had before the DML operation
    - :NEW - the values that the row will have after the DML operation completes
  - Without FOR EACH ROW
    - Fires once per DML statement, regardless of how many rows are affected
    - Does not have access to individual row values

Operation	:OLD	:NEW
INSERT	NULL (row did not exist)	The values being inserted
UPDATE	The values before the change	The values after the change
DELETE	The values being deleted	NULL (row will not exist)

```
CREATE OR REPLACE TRIGGER trg_salary_change
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    -- :OLD.salary is what the salary was before the update
    -- :NEW.salary is what it is being changed to
    IF :NEW.salary > :OLD.salary * 1.20 THEN
        INSERT INTO salary_change_alerts (
            employee_id,
            old_salary,
            new_salary,
            change_pct,
            changed_at
        ) VALUES (
            :NEW.employee_id,
            :OLD.salary,
            :NEW.salary,
            ROUND((:NEW.salary - :OLD.salary) / :OLD.salary * 100, 2),
            SYSTIMESTAMP
        );
    END IF;
END;
/
```

Triggers

- The audit requirement
  - Enterprise systems frequently require a complete, tamper-resistant record of who changed what data and when
  - Application-level audit logging can be bypassed
    - Direct SQL from DBA tools, ETL processes, migration scripts all bypass application code
    - A database trigger fires for every DML on the table, from any source
    - it cannot be bypassed without disabling the trigger itself, which requires elevated privileges and is itself auditable

```
-- The audit log table
CREATE TABLE employees_audit (
    audit_id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id    NUMBER         NOT NULL,
    action         VARCHAR2(10)   NOT NULL,   -- INSERT, UPDATE, DELETE
    changed_by     VARCHAR2(128)  NOT NULL,   -- database username
    changed_at     TIMESTAMP      NOT NULL,
    old_full_name  VARCHAR2(200),
    new_full_name  VARCHAR2(200),
    old_salary     NUMBER,
    new_salary     NUMBER,
    old_status     VARCHAR2(20),
    new_status     VARCHAR2(20),
    old_dept_id    NUMBER,
    new_dept_id    NUMBER
);
```

Triggers

- Audit trigger
  - AFTER trigger
    - Only records changes that actually committed to the row, not rejected writes
  - FOR EACH ROW
    - Captures the specific values that changed for each affected row
  - SYS_CONTEXT
    - Retrieves the actual database session user, not an application-level user ID
    - More reliable for forensic purposes

```
-- The audit trigger
CREATE OR REPLACE TRIGGER trg_employees_audit
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employees_audit (
        employee_id,
        action,
        changed_by,
        changed_at,
        old_full_name,  new_full_name,
        old_salary,     new_salary,
        old_status,     new_status,
        old_dept_id,    new_dept_id
    ) VALUES (
        COALESCE(:NEW.employee_id, :OLD.employee_id),
        CASE
            WHEN INSERTING THEN 'INSERT'
            WHEN UPDATING  THEN 'UPDATE'
            WHEN DELETING  THEN 'DELETE'
        END,
        SYS_CONTEXT('USERENV', 'SESSION_USER'),
        SYSTIMESTAMP,
        :OLD.full_name,   :NEW.full_name,
        :OLD.salary,      :NEW.salary,
        :OLD.status,      :NEW.status,
        :OLD.department_id, :NEW.department_id
    );
END trg_employees_audit;
/
```

Triggers

- Audit trigger
  - COALESCE on employee_id
    - For DELETE, :NEW is null; for INSERT, :OLD is null; COALESCE handles both
  - INSERTING, UPDATING, DELETING
    - Oracle predicates that identify which DML event fired the trigger

```
-- The audit trigger
CREATE OR REPLACE TRIGGER trg_employees_audit
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employees_audit (
        employee_id,
        action,
        changed_by,
        changed_at,
        old_full_name,  new_full_name,
        old_salary,     new_salary,
        old_status,     new_status,
        old_dept_id,    new_dept_id
    ) VALUES (
        COALESCE(:NEW.employee_id, :OLD.employee_id),
        CASE
            WHEN INSERTING THEN 'INSERT'
            WHEN UPDATING  THEN 'UPDATE'
            WHEN DELETING  THEN 'DELETE'
        END,
        SYS_CONTEXT('USERENV', 'SESSION_USER'),
        SYSTIMESTAMP,
        :OLD.full_name,   :NEW.full_name,
        :OLD.salary,      :NEW.salary,
        :OLD.status,      :NEW.status,
        :OLD.department_id, :NEW.department_id
    );
END trg_employees_audit;
/
```

Triggers

- Validation triggers
  - Enforces business rules that constraints cannot express
  - What constraints cannot do
    - CHECK constraints are evaluated in isolation. they can only examine the values in the current row
    - They cannot query other tables, compare values across rows, or apply conditional logic based on related data
  - Triggers fill this gap
    - They can execute arbitrary SQL
    - Call functions
    - Raise exceptions to prevent invalid writes

```
-- Example: prevent salary updates that exceed the department's budget
CREATE OR REPLACE TRIGGER trg_validate_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
DECLARE
    v_dept_budget    NUMBER;
    v_current_spend  NUMBER;
    v_available      NUMBER;
BEGIN
    -- Only validate if salary is being set or changed
    IF :NEW.salary IS NULL THEN
        RETURN;
    END IF;

    -- Retrieve department budget
    SELECT budget
    INTO   v_dept_budget
    FROM   departments
    WHERE  department_id = :NEW.department_id;

    -- Calculate current salary expenditure excluding this employee
    SELECT NVL(SUM(salary), 0)
    INTO   v_current_spend
    FROM   employees
    WHERE  department_id = :NEW.department_id
    AND    employee_id  != NVL(:NEW.employee_id, -1)
    AND    status        = 'ACTIVE';

    v_available := v_dept_budget - v_current_spend;

    IF :NEW.salary > v_available THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Salary of ' || :NEW.salary ||
            ' exceeds available department budget of ' || v_available
        );
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'Department not found: ' || :NEW.depar
END trg_validate_salary;
/
```

Triggers

- Validation triggers
  - RAISE_APPLICATION_ERROR
    - The correct mechanism for raising a user-defined error from a trigger or stored procedure
    - Error numbers must be in the range -20000 to -20999, Oracle reserves this range for application use
  - The error propagates to the caller exactly like a built-in Oracle error
    - The DML is rolled back, the caller receives the error code and message

```
-- Example: prevent salary updates that exceed the department's budget
CREATE OR REPLACE TRIGGER trg_validate_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
DECLARE
    v_dept_budget    NUMBER;
    v_current_spend  NUMBER;
    v_available      NUMBER;
BEGIN
    -- Only validate if salary is being set or changed
    IF :NEW.salary IS NULL THEN
        RETURN;
    END IF;

    -- Retrieve department budget
    SELECT budget
    INTO   v_dept_budget
    FROM   departments
    WHERE  department_id = :NEW.department_id;

    -- Calculate current salary expenditure excluding this employee
    SELECT NVL(SUM(salary), 0)
    INTO   v_current_spend
    FROM   employees
    WHERE  department_id = :NEW.department_id
    AND    employee_id  != NVL(:NEW.employee_id, -1)
    AND    status        = 'ACTIVE';

    v_available := v_dept_budget - v_current_spend;

    IF :NEW.salary > v_available THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Salary of ' || :NEW.salary ||
            ' exceeds available department budget of ' || v_available
        );
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'Department not found: ' || :NEW.depar
END trg_validate_salary;
/
```

PS/SQL Use Cases

Use Cases

- The case for PL/SQL
  - Complex multi-step operations that would require many round-trips from application code
  - Logic that must be enforced regardless of which application or tool accesses the data
  - Bulk data processing where set-based SQL can be exploited directly in the database engine
  - Audit and validation logic that must be tamper-resistant and universally applied
  - Operations on very large datasets where moving data to the application tier is impractical

Belongs in PL/SQL	Belongs in the application
Audit triggers	Business workflow logic
Cross-table validation	User interface rules
Bulk data processing	Domain model computation
Referential integrity enforcement	Presentation formatting
ETL and data transformation	Authentication and authorization

Use Cases

- The case against PL/SQL
  - Business logic that changes frequently
    - Stored procedures require database deployments to change, which adds friction to release cycles
  - Logic that is specific to one application and has no cross-cutting concern
    - It belongs in the application, not the database
  - Complex domain logic that benefits from unit testing frameworks, dependency injection, and modern software engineering practices
    - PL/SQL testing tooling exists but is less mature than application-tier testing
  - Logic that needs to scale horizontally
    - Stored procedures run in the database tier, which scales differently from application tiers

Belongs in PL/SQL	Belongs in the application
Audit triggers	Business workflow logic
Cross-table validation	User interface rules
Bulk data processing	Domain model computation
Referential integrity enforcement	Presentation formatting
ETL and data transformation	Authentication and authorization

Use Cases

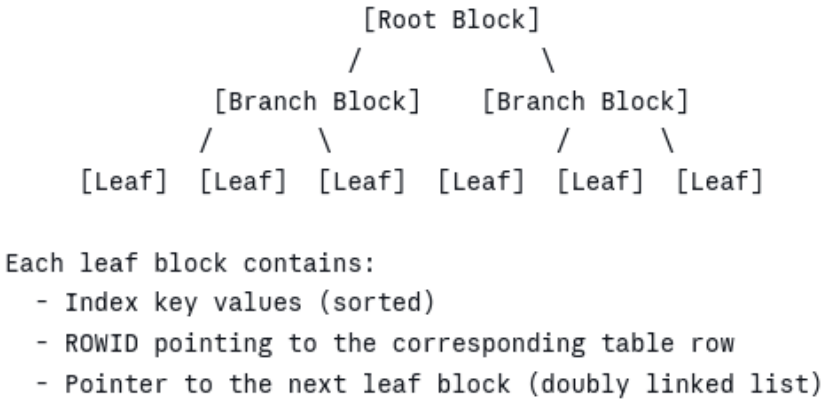
- Common mistakes
  - Putting everything in PL/SQL turns the database into a monolith
    - Schema changes, logic changes, and performance tuning become entangled
  - Putting nothing in PL/SQL and doing all data manipulation in the application tier
    - Sacrifices integrity enforcement, produces excessive round-trips, and makes audit logging bypassed

Belongs in PL/SQL	Belongs in the application
Audit triggers	Business workflow logic
Cross-table validation	User interface rules
Bulk data processing	Domain model computation
Referential integrity enforcement	Presentation formatting
ETL and data transformation	Authentication and authorization

Indexes

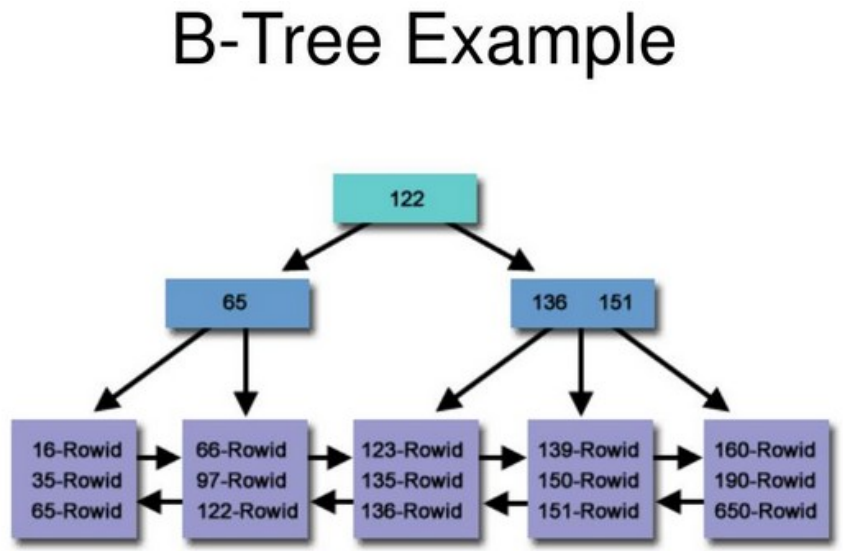
B-tree index

- B-tree stands for Balanced Tree
  - A B-tree index is a self-balancing tree structure that maintains sorted order across all indexed values
  - Oracle's standard index is a B-tree index and it is what is created by default with CREATE INDEX
  - It was originally developed by Rudolf Bayer and Edward McCreight in 1972
  - Remains the foundational index structure in virtually every relational database



B-tree index

- B-tree
  - The root and branch blocks contain key values and pointers to child blocks
  - All data is stored in the leaf blocks, which form a doubly linked list in key order
  - The tree is always balanced
    - Every path from root to leaf traverses the same number of levels
  - Balance is maintained automatically as rows are inserted and deleted

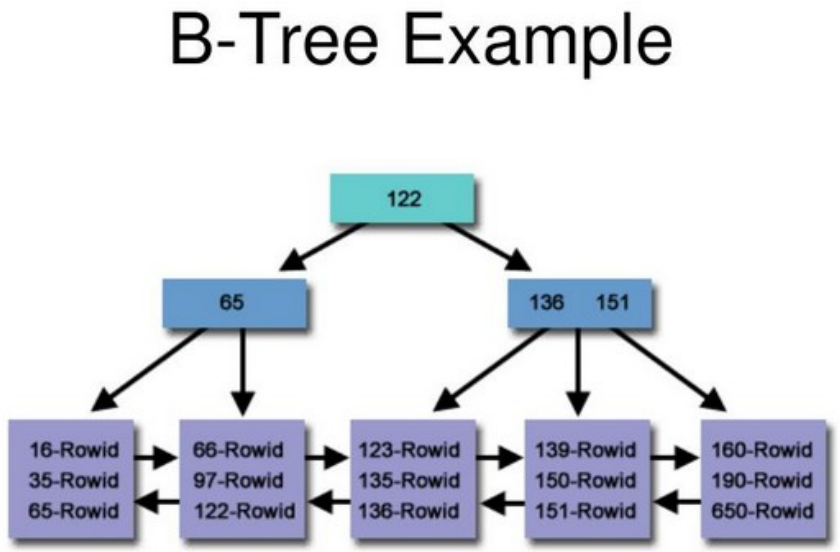


An Oracle B-Tree Index

Source (http://www.dba-oracle.com)

B-tree index

- Using the index to find a row
  - Start at the root block
  - Compare the search value to the key values in the root
  - Follow the pointer to the appropriate branch
  - Repeat down the branch levels until a leaf block is reached
  - Scan the leaf block to find the exact key value and its associated ROWID
  - Use the ROWID to fetch the actual row from the table

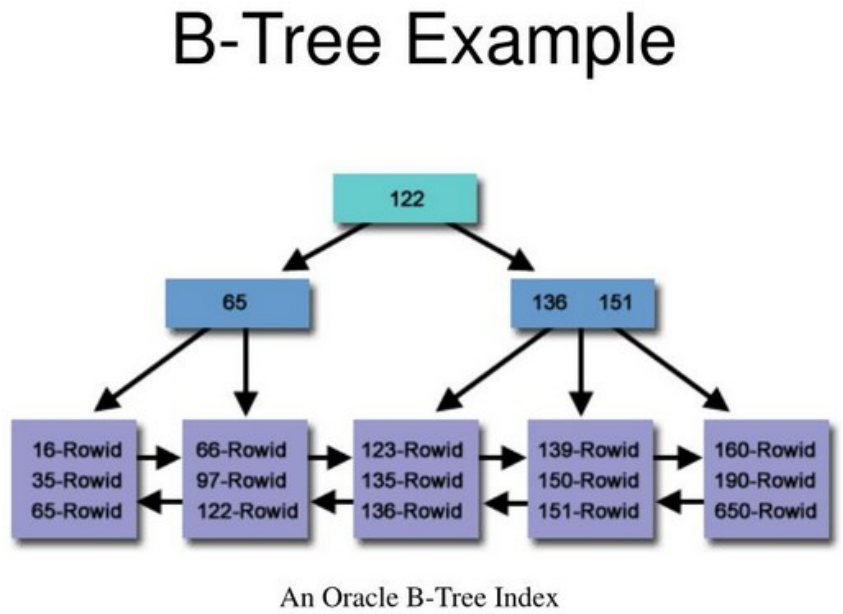


An Oracle B-Tree Index

Source (http://www.dba-oracle.com)

B-tree index

- What makes B-tree indexes fast
  - A table with one billion rows might have a B-tree index only four or five levels deep
  - Finding any row requires at most four or five block reads to traverse the tree, plus one block read to fetch the table row
  - This is independent of the table size
    - The depth grows logarithmically with the number of rows

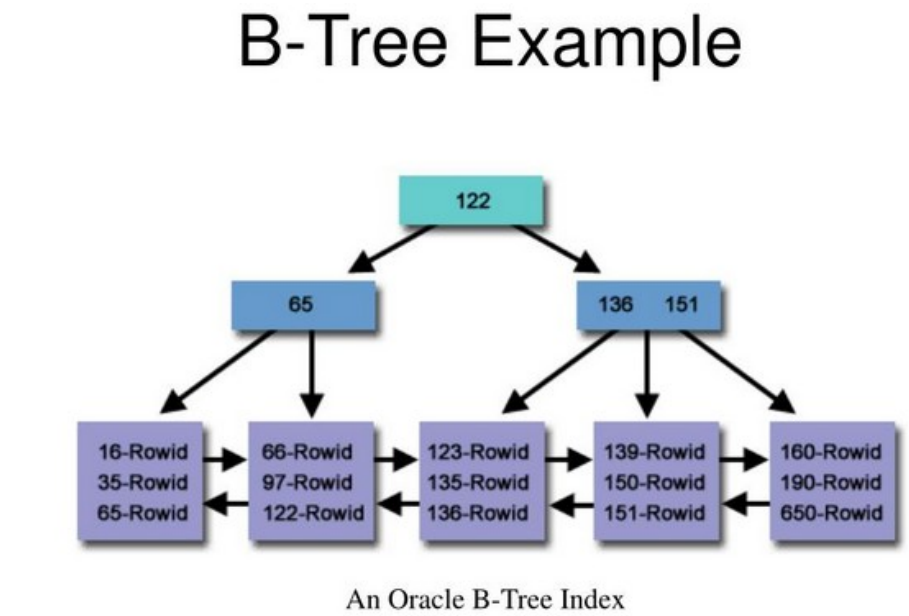


An Oracle B-Tree Index

Source (http://www.dba-oracle.com)

B-tree Index Variants

- Unique indexes
  - Enforces uniqueness in addition to providing fast lookup
  - Automatically created by Oracle for PRIMARY KEY and UNIQUE constraints
  - Can be created explicitly on any column combination that must be unique
  - Slightly faster than a non-unique index for equality lookups because Oracle stops at the first match

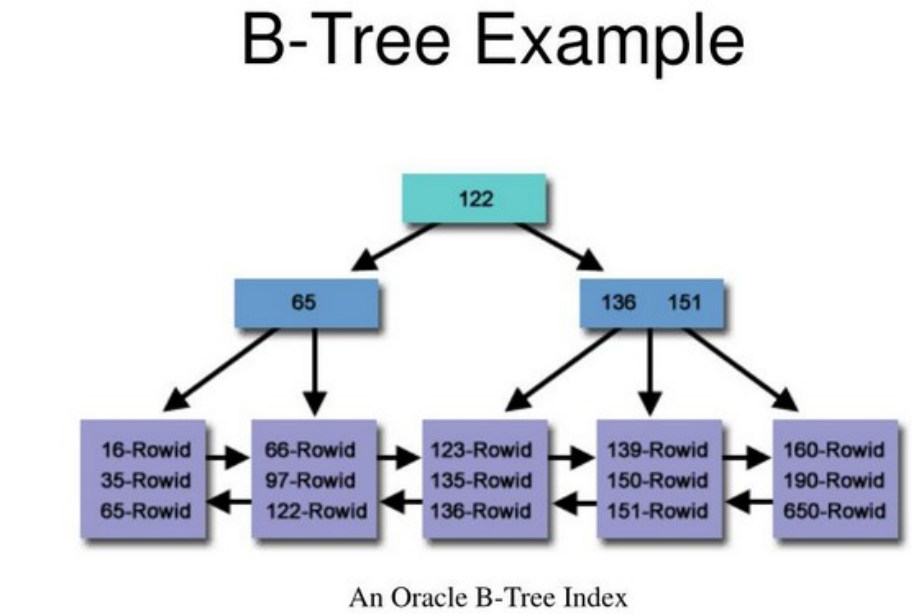


An Oracle B-Tree Index

Source (http://www.dba-oracle.com)

B-tree Index Variants

- Composite indexes
  - An index on two or more columns
  - Key values are sorted by the first column, then by the second within equal first-column values, and so on
  - Can satisfy queries that filter on any leading prefix of the indexed columns
  - Composite index on (department_id, status, hire_date)
    - Can be used for predicates on department_id alone
    - On (department_id, status)
    - Or on all three
    - But not on status alone or hire_date alone without department_id



An Oracle B-Tree Index

Source (http://www.dba-oracle.com)

Bitmap Indexes

- Problems with B-tree indexes
  - Underperform on low-cardinality columns
  - A B-tree index stores one entry per row
    - If a column has only three distinct values (ACTIVE, INACTIVE, PENDING) and one million rows, the index has one million entries
  - Most queries for a low-cardinality column return a large fraction of the table
    - 30% for PENDING for example
    - Makes the B-tree index less useful than a full table scan

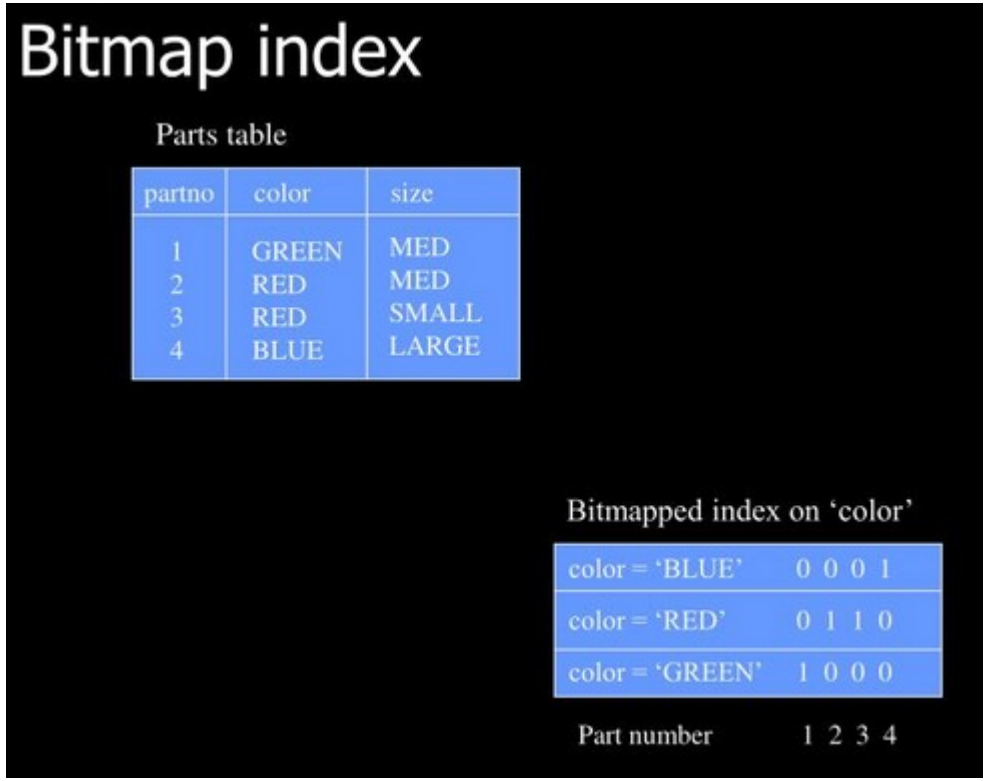


Image Credit: https://www.slideserve.com/anika/oracle-architecture

Bitmap Indexes

- Problems with B-tree indexes
  - Combining two low-cardinality predicates
    - WHERE status = 'ACTIVE' AND region = 'WEST'
    - B-tree indexes requires Oracle to merge two large ROWID sets, which is expensive
- What a bitmap index stores instead
  - For each distinct value of the indexed column
  - Oracle stores one bitmap
    - A sequence of bits, one per row in the table
    - Each bit is 1 if that row has that value, or 0 if it does not
  - A bitmap for ACTIVE in a one-million-row table
    - Is a string of one million bits, approximately 125KB, highly compressible

```
Column: status with values ACTIVE, INACTIVE, PENDING

Bitmap for ACTIVE:   1 0 0 1 1 0 1 0 1 1 ... (1 = this row is ACTIVE)
Bitmap for INACTIVE: 0 1 0 0 0 0 0 0 0 0 ...
Bitmap for PENDING:  0 0 1 0 0 1 0 1 0 0 ...
```

Bitmap Indexes

- The power of bitmap operations
  - Combining predicates is a bitwise AND, OR, or NOT operation on the bitmaps
    - Extremely fast in CPU
    - WHERE status = 'ACTIVE' AND region = 'WEST'
    - AND the two bitmaps together in a single CPU operation
  - The result bitmap identifies exactly which rows satisfy both conditions, without reading the table
  - Counting rows (SELECT COUNT(*)) can be answered entirely from the bitmap without touching the table at all

```
Column: status with values ACTIVE, INACTIVE, PENDING

Bitmap for ACTIVE:   1 0 0 1 1 0 1 0 1 1 ... (1 = this row is ACTIVE)
Bitmap for INACTIVE: 0 1 0 0 0 0 0 0 0 0 ...
Bitmap for PENDING:  0 0 1 0 0 1 0 1 0 0 ...
```

```
CREATE BITMAP INDEX idx_emp_status ON employees (status);
CREATE BITMAP INDEX idx_emp_region ON employees (region);

-- Oracle can combine these automatically
SELECT COUNT(*) FROM employees
WHERE  status = 'ACTIVE'
AND    region = 'WEST';
-- Resolved by AND-ing two bitmaps: no table access required
```

Bitmap Indexes

- The critical limitation
  - Bitmap Indexes belong only in read-heavy environments
- The locking problem
  - A B-tree index stores one entry per row
    - Updating a row's indexed value locks only that one index entry
  - A bitmap index stores one bitmap per distinct value that covers all rows
    - Updating any row's indexed value requires locking the entire bitmap for that value
  - If one session is updating a row's status from PENDING to ACTIVE
    - It holds a lock on the entire ACTIVE bitmap and the entire PENDING bitmap
    - Other attemo to update any row that has status = ACTIVE or status = PENDING waits for that lock to be released

```
Column: status with values ACTIVE, INACTIVE, PENDING

Bitmap for ACTIVE:   1 0 0 1 1 0 1 0 1 1 ... (1 = this row is ACTIVE)
Bitmap for INACTIVE: 0 1 0 0 0 0 0 0 0 0 ...
Bitmap for PENDING:  0 0 1 0 0 1 0 1 0 0 ...
```

```
CREATE BITMAP INDEX idx_emp_status ON employees (status);
CREATE BITMAP INDEX idx_emp_region ON employees (region);

-- Oracle can combine these automatically
SELECT COUNT(*) FROM employees
WHERE  status = 'ACTIVE'
AND    region = 'WEST';
-- Resolved by AND-ing two bitmaps: no table access required
```

Bitmap Indexes

- At OLTP concurrency levels
  - In a system with 100 concurrent users updating employee statuses
  - Each update locks a bitmap that covers hundreds of thousands of rows
  - Users queue behind each other even when updating completely different employees
  - What appears to be a simple single-row update becomes a serialization point that blocks the entire table
  - Throughput collapses under concurrent write load

```
Column: status with values ACTIVE, INACTIVE, PENDING

Bitmap for ACTIVE:   1 0 0 1 1 0 1 0 1 1 ... (1 = this row is ACTIVE)
Bitmap for INACTIVE: 0 1 0 0 0 0 0 0 0 0 ...
Bitmap for PENDING:  0 0 1 0 0 1 0 1 0 0 ...
```

```sql
CREATE BITMAP INDEX idx_emp_status ON employees (status);
CREATE BITMAP INDEX idx_emp_region ON employees (region);

-- Oracle can combine these automatically
SELECT COUNT(*) FROM employees
WHERE  status = 'ACTIVE'
AND    region = 'WEST';
-- Resolved by AND-ing two bitmaps: no table access required
```

Bitmap Indexes

- The rule is absolute
  - Bitmap indexes must never be used on tables that receive concurrent DML in production
  - The only safe environment for bitmap indexes is one where the data is loaded in bulk, the bitmap indexes are built after the load, and the table is then read-only or near-read-only until the next bulk load
  - This describes exactly the data warehouse pattern
    - Nightly or weekly bulk loads followed by read-only analytical querying

Factor	B-tree	Bitmap
Column cardinality	High (many distinct values)	Low (few distinct values)
DML pattern	Any -- handles concurrent updates safely	Read-only or bulk-load only
Query pattern	Point lookups and range scans	Multi-column AND/OR combinations, counts
Storage	One entry per row -- larger	One bit per row per value -- compact and compressible
Index combination	Expensive ROWID merge	Fast bitwise operation
Environment	OLTP	Data warehouse / OLAP

Partitions

Partitioning

- Partitioning
  - Partitioning divides a large table or index into smaller, physically separate pieces called partitions
  - Each partition is stored as its own segment, with its own extent allocation
  - To the application, the table appears as a single object
    - SQL queries reference the table name, not individual partitions
  - Oracle routes each row to the correct partition based on the value of the partition key column

Image Credit: https://oracle-base.com/articles/12c/partitioned-external-tables-12cr2

Partitioning

- Why partitioning exists
  - Very large tables
    - Hundreds of millions to billions of rows
    - Size create problems that indexes alone cannot solve
  - A full table scan of a billion-row table
    - Very expensive even if the query only needs rows from one month
  - An index on a billion-row table is large
    - Slow to maintain, and may still require reading many scattered blocks
  - Partitioning allows Oracle to eliminate entire partitions from consideration before reading a single row
    - Called partition pruning

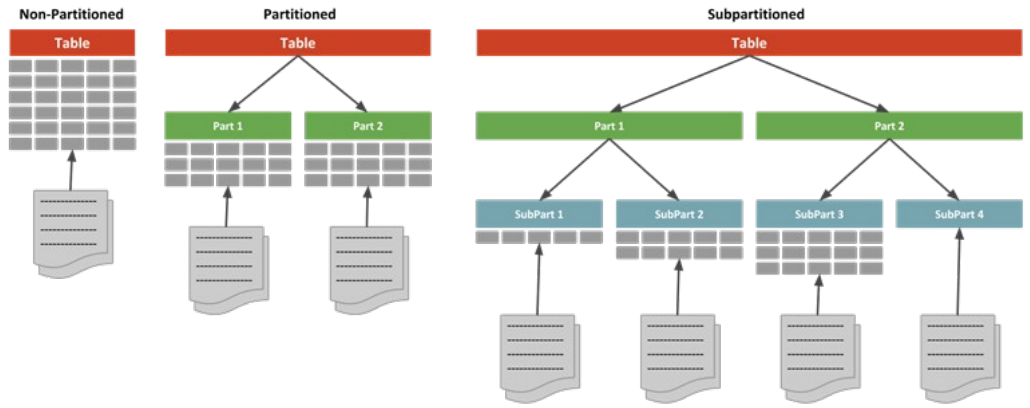


Image Credit: https://oracle-base.com/articles/12c/partitioned-external-tables-12cr2

Partitioning

- What partitioning does not do
  - Replace indexes
    - a query that filters on a non-partition-key column still needs an index within the relevant partition
  - Automatically improve all queries
    - Only queries that filter on the partition key benefit from partition pruning
- Partitioning does adds management complexity
  - Partition maintenance, local versus global indexes, partition exchange

```
-- Table partitioned by year on transaction_date
-- Query asking only for 2025 data

SELECT SUM(amount)
FROM   transactions
WHERE  transaction_date >= DATE '2025-01-01'
AND    transaction_date <  DATE '2026-01-01';

-- Without partitioning: Oracle reads all 10 years of data (10 billion rows)
-- With partitioning:    Oracle reads only the 2025 partition (1 billion rows)
-- The 9 other partitions are not touched at all
```

Partitioning

- Partition pruning
  - If a query's WHERE clause includes a predicate on the partition key column
  - Oracle can look at the predicate and determine which partitions could possibly contain matching rows.
  - It then reads only those partitions and ignores the rest entirely.
- Example
  - For a table partitioned by month across five years
    - A query asking for a single month's data reads 1/60th of the table.
  - This is not an index optimization, it is physical data elimination

```
-- Table partitioned by year on transaction_date
-- Query asking only for 2025 data

SELECT SUM(amount)
FROM   transactions
WHERE  transaction_date >= DATE '2025-01-01'
AND    transaction_date <  DATE '2026-01-01';

-- Without partitioning: Oracle reads all 10 years of data (10 billion rows)
-- With partitioning:    Oracle reads only the 2025 partition (1 billion rows)
-- The 9 other partitions are not touched at all
```

Partitioning

- Caveat
  - A query that does not filter on the partition key gets no pruning benefit
    - It must read all partitions
    - Which at large scale may be worse than a non-partitioned table because of the management overhead
  - The decision to partition a table, and the choice of partition key
    - Must be driven by the actual query patterns
    - The partition key should be the column that is most commonly used as a range filter in the most important queries

```
-- Table partitioned by year on transaction_date
-- Query asking only for 2025 data

SELECT SUM(amount)
FROM   transactions
WHERE  transaction_date >= DATE '2025-01-01'
AND    transaction_date <  DATE '2026-01-01';

-- Without partitioning: Oracle reads all 10 years of data (10 billion rows)
-- With partitioning:    Oracle reads only the 2025 partition (1 billion rows)
-- The 9 other partitions are not touched at all
```

Range Partitioning

- What range partitioning does
  - Dividing data by a Continuous value range
  - Assigns rows to partitions based on whether the partition key value falls within a specified range
  - The most common partitioning strategy for time-series data
    - Each partition holds one month, one quarter, or one year of data
  - Partition boundaries are defined by the DBA and can be extended over time as new periods arrive

```
CREATE TABLE transactions (
    transaction_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    account_id       NUMBER        NOT NULL,
    transaction_date DATE          NOT NULL,
    amount           NUMBER(15,2)  NOT NULL,
    category         VARCHAR2(50),
    status           VARCHAR2(20)
)
PARTITION BY RANGE (transaction_date) (
    PARTITION p_2023 VALUES LESS THAN (DATE '2024-01-01'),
    PARTITION p_2024 VALUES LESS THAN (DATE '2025-01-01'),
    PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01'),
    PARTITION p_future VALUES LESS THAN (MAXVALUE)
);
```

```
-- Split the catch-all MAXVALUE partition to add a specific 2026 partition
ALTER TABLE transactions SPLIT PARTITION p_future
    AT (DATE '2027-01-01')
    INTO (
        PARTITION p_2026,
        PARTITION p_future
```

Range Partitioning

- When range partitioning is appropriate
  - The table contains time-series data where queries almost always filter by date or date range
  - Old data has a different access pattern from new data
    - Less frequent, read-only
  - Data lifecycle management is required
    - Periodic archiving or purging of old data

```
-- Drop an entire year of old data in one instantaneous operation
-- No DELETE statement, no undo generation, no redo log volume
ALTER TABLE transactions DROP PARTITION p_2023;

-- Or archive it: move the partition to a different tablespace (cheap storage)
ALTER TABLE transactions MOVE PARTITION p_2023
    TABLESPACE archive_ts;
```

List Partitioning

- Dividing data by discrete values
  - Assigns rows to partitions based on whether the partition key value matches a specific list of values
  - Appropriate when the partitioning dimension is categorical rather than continuous
  - Common use cases
    - Partitioning by region, country, business unit, product category, or status

```
CREATE TABLE customer_interactions (
    interaction_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    customer_id      NUMBER       NOT NULL,
    region           VARCHAR2(20) NOT NULL,
    interaction_date DATE         NOT NULL,
    channel          VARCHAR2(20),
    outcome          VARCHAR2(50)
)
PARTITION BY LIST (region) (
    PARTITION p_north  VALUES ('NORTH', 'NORTHWEST', 'NORTHEAST'),
    PARTITION p_south  VALUES ('SOUTH', 'SOUTHWEST', 'SOUTHEAST'),
    PARTITION p_east   VALUES ('EAST'),
    PARTITION p_west   VALUES ('WEST'),
    PARTITION p_intl   VALUES ('EMEA', 'APAC', 'LATAM'),
    PARTITION p_other  VALUES (DEFAULT)
);
```

```
-- Only the p_north and p_east partitions are read
SELECT COUNT(*), AVG(response_time)
FROM   customer_interactions
WHERE  region IN ('NORTH', 'EAST')
AND    interaction_date > SYSDATE - 30;
```

List Partitioning

- The DEFAULT partition
  - Catches rows whose partition key value does not match any explicitly listed partition
  - Prevents insert failures when new values appear that were not anticipated at design time
  - Should be monitored
    - A growing DEFAULT partition may indicate that the list of values needs to be extended

```
CREATE TABLE customer_interactions (
    interaction_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    customer_id      NUMBER       NOT NULL,
    region           VARCHAR2(20) NOT NULL,
    interaction_date DATE         NOT NULL,
    channel          VARCHAR2(20),
    outcome          VARCHAR2(50)
)
PARTITION BY LIST (region) (
    PARTITION p_north  VALUES ('NORTH', 'NORTHWEST', 'NORTHEAST'),
    PARTITION p_south  VALUES ('SOUTH', 'SOUTHWEST', 'SOUTHEAST'),
    PARTITION p_east   VALUES ('EAST'),
    PARTITION p_west   VALUES ('WEST'),
    PARTITION p_intl   VALUES ('EMEA', 'APAC', 'LATAM'),
    PARTITION p_other  VALUES (DEFAULT)
);
```

```
-- Only the p_north and p_east partitions are read
SELECT COUNT(*), AVG(response_time)
FROM   customer_interactions
WHERE  region IN ('NORTH', 'EAST')
AND    interaction_date > SYSDATE - 30;
```

List Partitioning

- When list partitioning is appropriate
  - Queries are consistently filtered by a categorical column with a manageable number of distinct values
  - Different categories have significantly different access volumes
    - Hot regions versus cold regions
  - Data ownership or security requirements map to categorical values
    - A regional administrator should only see their region's data, for example

```
CREATE TABLE customer_interactions (
    interaction_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    customer_id      NUMBER       NOT NULL,
    region           VARCHAR2(20) NOT NULL,
    interaction_date DATE         NOT NULL,
    channel          VARCHAR2(20),
    outcome          VARCHAR2(50)
)
PARTITION BY LIST (region) (
    PARTITION p_north  VALUES ('NORTH', 'NORTHWEST', 'NORTHEAST'),
    PARTITION p_south  VALUES ('SOUTH', 'SOUTHWEST', 'SOUTHEAST'),
    PARTITION p_east   VALUES ('EAST'),
    PARTITION p_west   VALUES ('WEST'),
    PARTITION p_intl   VALUES ('EMEA', 'APAC', 'LATAM'),
    PARTITION p_other  VALUES (DEFAULT)
);
```

```
-- Only the p_north and p_east partitions are read
SELECT COUNT(*), AVG(response_time)
FROM   customer_interactions
WHERE  region IN ('NORTH', 'EAST')
AND    interaction_date > SYSDATE - 30;
```

Hash Partitioning

- Distributing data evenly
  - Used when there is no natural range or list
  - Applies a hash function to the partition key value and assigns the row to a partition based on the hash result
  - The distribution is controlled by Oracle's internal hash algorithm
    - The developer specifies only how many partitions to create
  - Produces an approximately even distribution of rows across all partitions regardless of the data values

```
-- Hash partition on a surrogate key: Oracle distributes rows evenly
CREATE TABLE audit_events (
    event_id       NUMBER GENERATED ALWAYS AS IDENTITY,
    entity_id      NUMBER        NOT NULL,
    event_type     VARCHAR2(50)  NOT NULL,
    event_date     TIMESTAMP     NOT NULL,
    event_payload  CLOB
)
PARTITION BY HASH (entity_id)
PARTITIONS 16;   -- number of partitions; should be a power of 2 for even distr
```

Hash Partitioning

- When to use Hash partitioning
  - When range and list do not apply
  - The partition key does not have a natural ordering or grouping that maps to a useful range or list
  - The data distribution is uneven and a list or range partition would produce some very large and some very small partitions
  - The goal is I/O parallelism rather than data pruning

```
-- Hash partition on a surrogate key: Oracle distributes rows evenly
CREATE TABLE audit_events (
    event_id       NUMBER GENERATED ALWAYS AS IDENTITY,
    entity_id      NUMBER         NOT NULL,
    event_type     VARCHAR2(50)   NOT NULL,
    event_date     TIMESTAMP      NOT NULL,
    event_payload  CLOB
)
PARTITION BY HASH (entity_id)
PARTITIONS 16;    -- number of partitions; should be a power of 2 for even distr
```

Hash Partitioning

- Advantages
  - Parallel query
    - Oracle can scan multiple partitions simultaneously using parallel query slaves
  - Reduced contention
    - Concurrent DML on the same table distributes across different partition segments, reducing block-level contention on high-insert workloads
  - I/O distribution
    - Partitions can be placed on different disk groups, distributing I/O load across storage

```
-- Hash partition on a surrogate key: Oracle distributes rows evenly
CREATE TABLE audit_events (
    event_id       NUMBER GENERATED ALWAYS AS IDENTITY,
    entity_id      NUMBER         NOT NULL,
    event_type     VARCHAR2(50)   NOT NULL,
    event_date     TIMESTAMP      NOT NULL,
    event_payload  CLOB
)
PARTITION BY HASH (entity_id)
PARTITIONS 16;   -- number of partitions; should be a power of 2 for even distr
```

Hash Partitioning

- What hash partitioning does not provide
  - Partition pruning
    - Oracle cannot determine which partition a specific value is in without applying the hash
    - There is no elimination of partitions based on a WHERE clause predicate on the hash key
  - Data lifecycle management
    - Hash partitions do not correspond to meaningful data groupings, so dropping or archiving a partition is not operationally meaningful

```
-- Hash partition on a surrogate key: Oracle distributes rows evenly
CREATE TABLE audit_events (
    event_id       NUMBER GENERATED ALWAYS AS IDENTITY,
    entity_id      NUMBER         NOT NULL,
    event_type     VARCHAR2(50)   NOT NULL,
    event_date     TIMESTAMP      NOT NULL,
    event_payload  CLOB
)
PARTITION BY HASH (entity_id)
PARTITIONS 16;    -- number of partitions; should be a power of 2 for even distr
```

Composite Partitioning

- Composite partitioning
  - Combining two partitioning strategies
  - A table is partitioned by one strategy, the primary partition
  - Each partition is further subdivided by a second strategy, the subpartition
  - Combines the benefits of two different partitioning approaches
  - The most common combination is range-hash or range-list
    - Range partition provides data lifecycle management (drop old years) and date-range pruning
    - Hash subpartition provides parallel I/O within each year and distributes insert contention

```
CREATE TABLE transactions (
    transaction_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    account_id       NUMBER        NOT NULL,
    transaction_date DATE          NOT NULL,
    amount           NUMBER(15,2)  NOT NULL,
    region           VARCHAR2(20)
)
PARTITION BY RANGE (transaction_date)
SUBPARTITION BY HASH (account_id)
SUBPARTITIONS 8
(
    PARTITION p_2024 VALUES LESS THAN (DATE '2025-01-01'),
    PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01'),
    PARTITION p_future VALUES LESS THAN (MAXVALUE)
);
```

Composite Partitioning

- Range-List
  - Time-based partitioning with categorical subdivision
  - A query filtering on both date and region prunes to a single subpartition
    - The most granular possible pruning
  - Each subpartition can be independently archived, moved, or managed

```
PARTITION BY RANGE (transaction_date)
SUBPARTITION BY LIST (region)
(
    PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01') (
        SUBPARTITION p_2025_north VALUES ('NORTH', 'NORTHEAST', 'NORTHWEST'),
        SUBPARTITION p_2025_south VALUES ('SOUTH', 'SOUTHEAST', 'SOUTHWEST'),
        SUBPARTITION p_2025_other VALUES (DEFAULT)
    ),
    PARTITION p_future VALUES LESS THAN (MAXVALUE) (
        SUBPARTITION p_fut_north VALUES ('NORTH', 'NORTHEAST', 'NORTHWEST'),
        SUBPARTITION p_fut_south VALUES ('SOUTH', 'SOUTHEAST', 'SOUTHWEST'),
        SUBPARTITION p_fut_other VALUES (DEFAULT)
    )
);
```

Composite Partitioning

- The management complexity trade-off
  - Composite partitioning produces many more physical segments than single-level partitioning
    - A range-list table with 10 years of data and 5 regions has 50 subpartitions plus the partition metadata
  - Each subpartition must be considered in index design, backup strategy, and maintenance operations
  - The additional complexity is justified only when both dimensions are genuinely used as query filters

```
PARTITION BY RANGE (transaction_date)
SUBPARTITION BY LIST (region)
(
    PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01') (
        SUBPARTITION p_2025_north VALUES ('NORTH', 'NORTHEAST', 'NORTHWEST'),
        SUBPARTITION p_2025_south VALUES ('SOUTH', 'SOUTHEAST', 'SOUTHWEST'),
        SUBPARTITION p_2025_other VALUES (DEFAULT)
    ),
    PARTITION p_future VALUES LESS THAN (MAXVALUE) (
        SUBPARTITION p_fut_north VALUES ('NORTH', 'NORTHEAST', 'NORTHWEST'),
        SUBPARTITION p_fut_south VALUES ('SOUTH', 'SOUTHEAST', 'SOUTHWEST'),
        SUBPARTITION p_fut_other VALUES (DEFAULT)
    )
);
```

Local and Global Indexes

- When a table is partitioned
  - Indexes on that table can be either local
    - One index partition per table partition
  - Or global
    - A single index structure spanning all partitions
  - Has significant implications for
    - Query performance
    - Partition maintenance operations
    - Index usability after partition changes

```
-- Local index: one index segment per table partition
CREATE INDEX idx_txn_account ON transactions (account_id) LOCAL;
```

```
-- Global index: one index structure for the entire table
CREATE INDEX idx_txn_account ON transactions (account_id) GLOBAL;

-- Drop partition with automatic global index update (slower but avoids invalid
ALTER TABLE transactions DROP PARTITION p_2022
    UPDATE GLOBAL INDEXES;
```

Local and Global Indexes

- Local indexes
  - Each table partition has its own corresponding index partition
  - Index partition and table partition are always aligned
    - The index partition contains only entries for rows in its corresponding table partition
  - Partition maintenance operations do not invalidate other index partitions
    - For example, DROP PARTITION, SPLIT PARTITION
  - A DROP PARTITION operation leaves all other local index partitions valid and usable

Factor	Local Index	Global Index
Queries filtered on partition key	Efficient -- pruning + local index	Efficient -- pruning eliminates partitions
Queries not filtered on partition key	Must probe all partition indexes	Single lookup -- more efficient
Partition maintenance (DROP, SPLIT)	Other partitions unaffected	Entire index invalidated unless UPDATE GLOBAL INDEXES
Index uniqueness across entire table	Cannot guarantee without partition key in index	Naturally unique across all partitions
Management simplicity	Simpler -- maintenance operations are safe	More complex -- maintenance requires index rebuild

Local and Global Indexes

- Global indexes
  - A single index structure spans all table partitions
  - Provides better performance for queries that do not filter on the partition key
    - The index can locate rows in any partition from a single lookup
  - Partition maintenance operations invalidate the entire global index
    - It must be rebuilt after each partition operation unless UPDATE INDEXES is specified

Factor	Local Index	Global Index
Queries filtered on partition key	Efficient -- pruning + local index	Efficient -- pruning eliminates partitions
Queries not filtered on partition key	Must probe all partition indexes	Single lookup -- more efficient
Partition maintenance (DROP, SPLIT)	Other partitions unaffected	Entire index invalidated unless UPDATE GLOBAL INDEXES
Index uniqueness across entire table	Cannot guarantee without partition key in index	Naturally unique across all partitions
Management simplicity	Simpler -- maintenance operations are safe	More complex -- maintenance requires index rebuild

Questions?